



Introdução

Registradores RISC-V

Registradores RISC-V



Dentre os registradores de **x1** a **x31**, alguns têm usos especiais em convenções de chamada de função ou pelo sistema operacional:

- **x1 (ra)**: Registrador de retorno (return address) para armazenar o endereço de retorno de chamadas de função.

x2 (sp): Ponteiro de pilha (stack pointer), usado como o topo da pilha em convenções de chamada.

x3 (gp): Ponteiro global (global pointer), usado para acessar dados globais.

x4 (tp): Ponteiro de thread (thread pointer), usado para dados específicos do thread.

x5-x7, x10-x17, x28-x31: Registradores temporários ou para passagem de argumentos em chamadas de função.

Registradores RISC-V

Continuação:

- **x8 (s0/fp):** Ponteiro de frame ou salvo, preservado entre chamadas de função.
- **x9 (s1):** Salvo, preservado entre chamadas de função.
- **x10-x11 (a0-a1):** Argumentos de função e valores de retorno para funções.
- **x12-x17 (a2-a7):** Argumentos de função.
- **x18-x27 (s2-s11):** Salvos, preservados entre chamadas de função.
- **x28-x31 (t3-t6):** Temporários, não preservados entre chamadas de função.

Instruções RISC-V

Instruções RISC-V

Na arquitetura RISC-V, as instruções são organizadas em vários formatos, dependendo do tipo de operação que realizam.

Esses formatos são projetados para simplificar a decodificação das instruções pelo processador, mantendo a flexibilidade para suportar uma ampla gama de operações.

As instruções em RISC-V têm um tamanho fixo de 32 bits, e a divisão desses bits varia de acordo com o formato da instrução.

Formato I (Tipo Immediato)



Campos:

- **opcode (7 bits):** Define o tipo de operação, indicando que a instrução utiliza um valor imediato em sua execução.
- **rd (5 bits):** O registrador destino onde o resultado da operação será salvo.
- **funct3 (3 bits):** Auxilia na determinação da operação específica a ser realizada, junto ao opcode.
- **rs1 (5 bits):** O registrador fonte da operação.
- **imm (12 bits):** O valor imediato utilizado na operação. Este valor pode ser usado para adições, comparações ou como um offset para operações de carga e armazenamento.

Instruções RISC-V

Formato S (Tipo Store)



Para instruções de armazenamento de dados na memória.

Campos:

- **opcode (7 bits):** Indica que a instrução é uma operação de armazenamento de dados na memória.
- **imm (12 bits, dividido):** O valor imediato que especifica o offset de endereçamento na memória a partir do endereço base fornecido pelo rs1. Dividido entre os campos superior e inferior para codificação.
- **funct3 (3 bits):** Determina o tamanho da unidade de dados a ser armazenada (por exemplo, byte, halfword, word).
- **rs1 (5 bits):** O registrador que contém o endereço base para o armazenamento de dados.
- **rs2 (5 bits):** O registrador que contém os dados a serem armazenados na memória.

Instruções RISC-V

Formato B (Tipo Branch)



Para instruções de desvio condicional baseadas na comparação de dois registradores.

Campos:

- **opcode (7 bits):** Especifica que a instrução é um desvio condicional.
- **imm (12 bits, dividido):** O valor imediato que representa o offset de desvio caso a condição seja verdadeira. Este valor é dividido e codificado em vários campos.
- **funct3 (3 bits):** Define a condição de desvio (por exemplo, igualdade, menor que).
- **rs1 e rs2 (5 bits cada):** Os registradores cujos valores serão comparados para determinar se o desvio será tomado.

Assembly RISC-V

Organização de um Programa em Assembly RISC-V: Seção de Dados



A seção de dados é usada para declarar variáveis estáticas e inicializar constantes que seu programa irá usar.

Variáveis e constantes são alocadas aqui com nomes simbólicos, facilitando o acesso no restante do programa.

Código

```
.data

msg: .asciz "Ola, mundo!\n" #Declara variavel do tipo string

numero: .word 123 #Declara variavel do tipo inteiro
```

Assembly RISC-V

Organização de um Programa em Assembly RISC-V: Seção de Dados



A seção de texto contém o código executável do programa. É aqui que as instruções do programa são escritas.

Esta seção começa com a diretiva **.text** e geralmente inclui uma etiqueta (label) para o ponto de entrada do programa, frequentemente denominado main em programas simples.

Código

```
.text
li a7 4 #Carrega codigo de chamada de sistema PrintString

la a0 msg #Carrega a string armazenada no endereco de msg

ecall #Realiza a chamada de sistema
```


Assembly RISC-V

Organização de um Programa em Assembly RISC-V: Instruções



Aqui você coloca as instruções que seu programa executará, começando pelo ponto de entrada.

Isso inclui operações lógicas e aritméticas, controle de fluxo, chamadas de função e manipulação de dados.

Código

```
main:
    li a7 4 #Carrega código de chamada de sistema PrintString

    la a0 msg #Carrega a string armazenada no endereço de msg

    ecall #Realiza a chamada de sistema
```


Assembly RISC-V

Chamadas de Sistema



Os passos para invocar uma chamada de sistemas são os seguintes:

- 1 **Definir o Número da Chamada de Sistema:** Cada chamada de sistema tem um número único associado. Esse número deve ser colocado em um registrador específico. Em sistemas Linux sobre RISC-V, o número da chamada de sistema é colocado no registrador **a7**.
- 2 **Configurar Argumentos:** Se a chamada de sistema requer argumentos, eles devem ser colocados nos registradores **a0** a **a6**, conforme a convenção de chamadas do sistema operacional. Por exemplo, **a0** pode ser usado para um descritor de arquivo, **a1** para o endereço de um buffer de dados, e assim por diante.

Operações Aritméticas



Operações Aritméticas

Continuação:

- **Operações com Números de Ponto Flutuante:**
 - **Precisão Simples (Extensão "D"):**
 - **fadd.d:** Soma de ponto flutuante de precisão dupla.
 - **fsub.d:** Subtração de ponto flutuante de precisão dupla.
 - **fmul.d:** Multiplicação de ponto flutuante de precisão dupla.
 - **fdiv.d:** Divisão de ponto flutuante de precisão dupla.
 - **fsqrt.d:** Raiz quadrada de ponto flutuante de precisão dupla.
 - **fmin.d:** Mínimo de dois números de ponto flutuante de precisão dupla.
 - **fmax.d:** Máximo de dois números de ponto flutuante de precisão dupla.

Exemples 1

Assembly RISC-V

Exemplos 1



Somando dois números

```
.data
msg_1: .asciz "Informe o primeiro numero: "
msg_2: .asciz "Informe o segundo numero: "
resultado: .asciz "Resultado: "
.text
.global main
main:
    li a7 4
    la a0 msg_1
    ecall #Imprime mensagem para primeiro numero
    li a7 5
    ecall #Leitura do teclado do segundo numero
    mv t0 a0 #Copia o valor de a0 e t0
    li a7 4
    la a0 msg_1
    ecall #Imprime mensagem para segundo numero
    li a7 5
    ecall #Leitura do teclado do segundo numero
    mv t1 a0 #Copia o valor de a0 e t1
    ...
```


Desvios Condicionais

Assembly RISC-V

Desvios Condicionais



Aqui estão as principais operações aritméticas em Assembly RISC-V, juntamente com uma breve descrição do que cada uma faz:

• Operações com Inteiros:

- **beq:** Branch if Equal - Desvia se dois registradores são iguais.
- **bne:** Branch if Not Equal - Desvia se dois registradores não são iguais.
- **blt:** Branch if Less Than - Desvia se o primeiro registrador é menor que o segundo (sinalizado).
- **bge:** Branch if Greater Than or Equal - Desvia se o primeiro registrador é maior ou igual ao segundo (sinalizado).
- **bltu:** Branch if Less Than Unsigned - Desvia se o primeiro registrador é menor que o segundo (não sinalizado).
- **bgeu:** Branch if Greater Than or Equal Unsigned - Desvia se o primeiro registrador é maior ou igual ao segundo (não sinalizado).

Exemples 2

Assembly RISC-V

Exemplos 2



Comparando números (cont.)

```
li a7 5
ecall
mv t1 a0

blt t1 t0 t0_maior_t1 #Verifica se t0 e maior que t1
blt t0 t1 t0_menor_t1 #Verifica se t0 e maior que t1
j t0_igual_t1

t0_maior_t1:
li a7 1
mv a0 t0
ecall

li a7 4
la a0 resultado_1
ecall

li a7 1
mv a0 t1
ecall
```



Assembly RISC-V

Exemplos 2



Comparando números (cont.)

```
t0_igual_t1:
    li a7 1
    mv a0 t0
    ecall

    li a7 4
    la a0 resultado_2
    ecall

    li a7 1
    mv a0 t1
    ecall

    j sair

sair:
    li a7 10
    ecall
```

Exercícios 1

Assembly RISC-V

Exercícios 1



- Crie um programa que simula um controle de direção básico com quatro direções (Norte=0, Sul=1, Leste=2, Oeste=3). Para uma entrada de direção, o programa deve indicar a ação correspondente (por exemplo, "Mover para o norte").
- Implemente um programa que verifique se um número fornecido é uma potência de dois. O programa deve armazenar 1 em um registrador se o número for uma potência de dois, e 0 se não for.
- Escreva um programa que calcule o discriminante (b^2-4ac) de uma equação quadrática e indique se as raízes são reais e distintas, reais e iguais, ou complexas.

Assembly RISC-V

Exercícios 1



- Desenvolva um programa que converta a temperatura de Celsius para Fahrenheit e vice-versa, baseado em um código de operação. Inclua verificações para entradas válidas.
- Implemente um programa que teste se um número n é divisível por outro número m , sem deixar resto. Utilize operações aritméticas para encontrar o resto da divisão.
- Crie um programa que classifique um caractere (valor ASCII) como letra maiúscula, letra minúscula, dígito ou símbolo especial. Utilize a tabela ASCII e desvios condicionais para a classificação.

Exercícios 1



- Escreva um programa que, dadas três notas, calcule a média e a classifique em categorias ("Excelente" para médias acima de 90, "Bom" para médias entre 70 e 89, "Satisfatório" para médias entre 50 e 69, e "Insuficiente" para médias abaixo de 50).

Operações Lógicas



Exemplo

```
.data

.text

.global main

main:
    li t0 14 #1110
    andi t1 t0 13 #1101
    mv a0 t1
    li a7 1
    ecall #Imprime o inteiro 12

    ori t1 t0 1#0001
    mv a0 t1
    li a7 1
    ecall #Imprime o inteiro 15

    xori t1 t0 13
    mv a0 t1
    li a7 1
```


Operações de Deslocamento de Bits



As instruções de deslocamento de bits são essenciais na arquitetura RISC-V, permitindo a manipulação eficiente de dados ao nível de bit.

Essas operações são amplamente utilizadas para otimização de cálculos matemáticos, manipulação de campos de bits, e outras operações de baixo nível que requerem precisão ao nível de bit.

Em RISC-V, existem várias instruções de deslocamento, cada uma adequada para diferentes casos de uso. As instruções podem ser categorizadas em deslocamentos lógicos e aritméticos, e podem operar com valores imediatos (fixos) ou variáveis (contidos em registradores).

Operações de Deslocamento



Exemplo

```
.data

.text

.global main

main:
    li t0 4
    slli t1 t0 3 #Multiplica 4 por 8 (2 elevado a 3)
    li a7 1
    mv a0 t1
    ecall #Imprime o inteiro 32

    mv t0 t1
    srli t1 t0 2 #Divide 32 por 4 (2 elevado a 2)
    li a7 1
    mv a0 t1
    ecall #Imprime o inteiro 8
```


Assembly RISC-V

Repetições: Exemplo de Implementação WHILE



Implementação do WHILE

```
.data

.text

.global main

main:
    li t0 10

while_loop:
    # Verifica a condicao do loop: se t0 <= 0, saia do loop.
    blez t0, end_while # Se t0 for menor ou igual a zero, pule para o
    fim.
    # Corpo do loop: (faca algo aqui).
    # Por exemplo, vamos apenas decrementar t0 por agora.
    addi t0, t0, -1
    # Volta ao inicio do loop para verificar a condicao novamente.
    j while_loop

end_while:
```


Assembly RISC-V

Repetições: Exemplo de Implementação FOR



Implementação do FOR

```
.data

.text

.global main

main:
    li t0 0

for_loop:
    li t1, 10      # t1 e o limite do nosso loop, 10 neste caso
    bge t0, t1, end_for # Condicao de Continuacao: Se t0 >= 10, saia do
                        # loop
    # Corpo do loop: (faca algo aqui)
    addi t0, t0, 1 # Incremento: t0 = t0 + 1
    j for_loop     # Salto incondicional de volta para o inicio do loop

end_for:
    # 0 codigao aqui sera executado apos a conclusao do loop
```


Assembly RISC-V

Funções



Exemplo de Implementação de uma Função Soma

```
.data

.text

.global main

main:
    # Prepara os argumentos para a funcao soma
    li a0, 5          # Primeiro argumento: 5
    li a1, 10         # Segundo argumento: 10
    # Chama a funcao soma
    jal soma

    li a7, 93         # Codigo da syscall para terminar o programa (exit)
    ecall

# Funcao soma
# Assume que os argumentos estao em a0 e a1
# Retorna o resultado em a0
soma:
    add a0, a0, a1    # Soma os argumentos a0 e a1, resultado vai para a0
```


Funções: Chamadas Aninhadas de Funções

- **Função Ativa:**

Uma função torna-se ativa quando é chamada e continua sendo a função ativa até que complete sua execução e retorne ao chamador.

Durante seu período ativo, a função possui controle total sobre os registradores especificados (de acordo com a convenção de chamada) e a parte da pilha que alocou para seu uso.

Quando uma função (**função A**) chama outra função (**função B**), **função A** se torna a função chamadora e **função B** a função chamada ou função ativa.

Isso cria uma pilha de chamadas, onde **função B** será a função ativa até que conclua sua execução e retorne a **função A**.

Funções: Chamadas Aninhadas de Funções

- **Função Ativa:**

A função ativa geralmente aloca espaço na pilha para armazenar variáveis locais, salvar os valores dos registradores que são modificados durante sua execução, e para possíveis argumentos que excedam os registradores disponíveis para passagem de parâmetros.

Este espaço na pilha é dedicado exclusivamente à função ativa e é desalocado quando a função completa sua execução e o controle retorna à função chamadora.

Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



- **Passagem de Argumentos:**

Em RISC-V, a passagem de argumentos segue uma convenção de chamada definida que especifica como os argumentos são passados das funções chamadoras para as chamadas. As principais características dessa convenção incluem:

- **Registradores de Argumentos:**

- RISC-V usa os registradores **a0** até **a7** (x10 até x17) para passar os primeiros oito argumentos inteiros ou ponteiros às funções.
- Os argumentos são colocados nesses registradores pela função chamadora antes de fazer a chamada.
- **a0** também é usado para retornar valores da função. Se uma função retorna um valor, esse valor é colocado em **a0**.

●			

Funções: Chamadas Aninhadas de Funções

Continuação:

Funções: Chamadas Aninhadas de Funções



Exemplo de Passagem de Parâmetros por Valor

```
.data

.text

.global main

main:
    ...
    li a0 5
    jal pow2
    ...

pow2:
    mul a0 a0 a0
    ret
```


Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



Exemplo de Passagem de Parâmetros por Referência

```
.data
    x: .word 5
    y: .word 1
.text
.global main
main:
    ...
    la t1 x
    la t2 y
    mv a0 t1
    mv a1 t2
    jal troca
    ...

troca:
    lw t0 (a0)
    lw t1 (a1)
    sw t0 (a1)
    sw t1 (a0)
    ret
```





- **Preservação de Estado:**

Se esses valores originais não forem restaurados após a execução da função, isso pode levar a resultados incorretos, comportamentos inesperados e bugs difíceis de rastrear.

Portanto, preservar o estado permite que cada função opere de maneira isolada, sem interferir nos resultados de outras partes do programa.

Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



- **Preservação de Estado:**

- **Registradores de Salvamento de Chamador (Caller-Saved):**

- Esses registradores, quando usados dentro de uma função que os altera, devem ser salvos (tipicamente na pilha) pelo chamador se ele deseja preservar seus valores originais após a chamada da função.
- Em RISC-V, os registradores **a0-a7** (usados para argumentos e retornos de funções) e **t0-t6** (registradores temporários) são considerados caller-saved.

- **Registradores de Salvamento de Chamado (Callee-Saved):**

- São registradores que, se modificados por uma função, devem ser salvos e restaurados por essa função antes de retornar ao chamador.
- Em RISC-V, os registradores **s0-s11** são callee-saved. Isso significa que qualquer função que utilize esses registradores para operações durante sua execução deve salvá-los no início da função e restaurá-los antes de retornar.

Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



Exemplo de Preservação de Estados

```
.data

.text

.global main

main:
    li a0 5 #a
    li a1 3 #b
    li a2 4 #c
    li a3 2 #d

    addi sp sp -4 #Reserva espaco na pilha para a3
    sw a3 0(sp) #Salva o valor de a3 no topo da pilha

    jal calcular

    lw a3 0(sp) #Restaura o valor original de a3 do topo da pilha
    addi sp sp 4 #Libera espaco na pilha

    li a7 93 #Syscall para sair do programa
```


Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



Exemplo de Preservação de Estados

calcular:

```
addi sp sp -16 #Aloca 4 espacos para registradores
sw ra 12(sp) #Salva ra no da pilha
sw a0 8(sp) #Salva a0 na pilha
sw a1 4(sp) #Salva a1 na pilha
sw a2 0(sp) #Salva a2 na pilha
```

```
jal somar #Chamada da funcao somar
```

```
mv a1 a2 #Passagem de parametro para multiplicar
jal multiplicar #Chamada da funcao multiplicar
```

```
mv a1 a3 #Passagem de parametro para substituir
jal subtrair #Chamada da funcao subtrair
```

```
lw a2 0(sp) #Restaura a2 da pilha
lw a1 4(sp) #Restaura a1 da pilha
lw a0 8(sp) #Restaura a0 da pilha
lw ra 12(sp) #Restaura endereco de retorno
addi sp sp 16 #Libera 4 espacos na pilha
```

Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



Exemplo de Preservação de Estados

somar:

```
addi sp sp -4 #Aloca 1 espaço na pilha
sw ra 0(sp) #Salva o ponto de retorno na pilha
```

```
add a0 a0 a1 #Soma a0 e a1 e salva em a0
```

```
lw ra 0(sp) #Restaura endereço de retorno
addi sp sp 4 #Libera um espaço na pilha
ret #Retorna para o ponto de chamada de calcular
```

multiplicar:

```
addi sp sp -4 #Aloca 1 espaço na pilha
sw ra 0(sp) #Salva o ponto de retorno na pilha
```

```
mul a0 a0 a1 #Multiplica a0 e a1 salva em a0
```

```
lw ra 0(sp) #Restaura endereço de retorno
addi sp sp 4 #Libera 1 espaço na memória
ret #Retorna para o ponto de chamada de calcular
```



Funções: Chamadas Aninhadas de Funções

Exemplo de Preservação de Estados

subtrair:

```
addi sp sp -4 #Aloca 1 espaco na pilha
```

```
sw ra 0(sp) #Salva o ponto de retorno na pilha
```

```
sub a0 a0 a1 #Subtrai a0 e a1 e salva em a0
```

```
lw ra 0(sp) #Restaura endereço de retorno
```

```
addi sp sp 4 #Libera 1 espaço na memória
```

```
ret #Retorna para o ponto de chamada de calcular
```

Funções: Chamadas Aninhadas de Funções

- Chamada de Função:

A instrução para chamar uma função é **jal** (*Jump And Link*).

jal salva o endereço de retorno no registrador **ra** (registrador de retorno) e depois salta para o endereço da função chamada.

O endereço de retorno é o ponto logo após a chamada da função, onde o controle do programa deve retornar após a conclusão da função.

Assembly RISC-V

Funções: Chamadas Aninhadas de Funções



• Retorno de Funções:

Em RISC-V, quando uma função é chamada usando a instrução **jal**, o endereço logo após a chamada é automaticamente armazenado no registrador **ra**.

Isso prepara o programa para saber exatamente onde retornar após a conclusão da função.

A instrução **ret** é um pseudônimo para a instrução **jalr zero 0(ra)**, que faz com que o programa salte para o endereço armazenado em **ra**.

Essa instrução é usada no final da função para devolver o controle ao chamador.

Funções: Chamadas de Funções Recursivas

Uma função recursiva é uma função que chama a si mesma durante sua execução.

Esse tipo de função é comumente usada em programação para resolver problemas que podem ser divididos em subproblemas mais simples de natureza similar.

A recursividade é uma ferramenta poderosa, especialmente útil para trabalhar com estruturas de dados que possuem uma natureza recursiva, como árvores e grafos, ou para implementar algoritmos como busca e ordenação.

●			

Funções: Chamadas de Funções Recursivas

```
.data
.text
.global main

main:
    li a0, 5          # Carrega o numero 5 em a0, para calcular 5!
    jal ra, factorial # Chama a funcao factorial e salva o endereco
de retorno em ra

    li a7, 93
    ecall
```



Assembly RISC-V

Funções: Chamadas de Funções Recursivas

Exemplo de Função Recursiva

factorial:

```
    addi sp, sp, -8      # Decrementa o ponteiro de pilha para salvar
espaco para ra e a0
```

```
sw ra, 4(sp)    # Salva o conteudo do registrador ra na pilha
```

```
sw a0, 0(sp)    # Salva o conteúdo do registrador a0 na pilha
```

```
li a1, 1          # Preparar para testar se n == 1
```

```
    beq a0, a1, base_case # Se sim, tratar o caso base (fatorial de
```

```
li a1, 0
```

```
beq a0, a1, base_case # Se n == 0, tratar tambem como caso base
```

```
    addi a0, a0, -1    # Preparar para a chamada recursiva: calcular
factorial(n-1)
```

```
jal ra, factorial # Chamada recursiva
```

```
lw a1, 0(sp)      # Recuperar o valor original de n
```

```
mul a0, a0, a1    # n * factorial(n-1)
```

```
lw ra, 4(sp)      # Restaurar o valor original de ra
```

```
addi sp, sp, 8    # Restaurar o ponteiro da pilha
```


Funções: Chamadas de Funções Recursivas

Exercícios 2

Assembly RISC-V

Exercícios 2



- 1 Escreva um programa em Assembly RISC-V que conta progressivamente de 1 até um número N fornecido. O programa deve imprimir cada número da contagem. Para este exercício, assuma que existe uma maneira de imprimir um número no seu ambiente.
- 2 Crie um programa em Assembly RISC-V que calcula a soma dos números de 1 até N, onde N é um valor fornecido. O programa deve armazenar o resultado da soma e, ao final, imprimir esse resultado.
- 3 Desenvolva um programa em Assembly RISC-V que verifica se um número N fornecido é primo. O programa deve imprimir uma mensagem indicativa se o número é primo ou não. Lembre-se, um número primo é um número maior que 1 que não tem divisores positivos além de 1 e ele mesmo.

Assembly RISC-V

Exercícios 2



- 4 Implemente um programa em Assembly RISC-V que calcula o fatorial de um número N fornecido (ou seja, $N!$). O fatorial de um número é o produto de todos os inteiros positivos menores ou iguais a ele. Por exemplo, o fatorial de 5 ($5!$) é $5 \times 4 \times 3 \times 2 \times 1 = 120$.
- 5 Escreva um programa em Assembly RISC-V que calcula o Máximo Divisor Comum (MDC) de dois números, A e B, fornecidos. O MDC de dois números é o maior número que divide ambos sem deixar resto. Você pode usar o algoritmo de Euclides, que repete a operação de subtração ou divisão modular até encontrar o MDC.

Assembly RISC-V

Vetores



Exemplo de Implementação de um Vetor

```
.data
    numeros: .space 40 #vetor de tamanho 10
    msg_1: .asciz "Informe o numero:"
.text
.global main
main:
    la t0 numeros #carregar o endereco do vetor
    li t1 0 #inicializa o indice do vetor

leitura:
    li a7 4
    la a0 msg_1
    ecall
    li a7 5
    ecall
    sw a0 0(t0) #armazena numero na posicao
    addi t0 t0 4 #incrementa endereco
    addi t1 t1 1 #incrementa indice
    li t2 10
    blt t1 t2 leitura #continua se t1 < 10
```

Assembly RISC-V

Vetores



Exemplo de Implementação de um Vetor

```
...
#saiu do loop
la t0 numeros #carregar o
li t1 0

impressao:
lw t3 0(t0) #carrega numero da posicao para registrador
li a7 1
mv a0 t3
ecall
addi t0 t0 4 #incrementa endereco
addi t1 t1 1 #incrementa indice
li t2 10
blt t1 t2 impressao #continua se t1 < 10
```


Assembly RISC-V

Exercícios 3



- ① Dados dois vetores de 10 inteiros cada, crie um terceiro vetor que seja a soma elemento a elemento dos dois primeiros.
- ② Encontre e armazene o valor do maior elemento de um vetor de 15 inteiros.
- ③ Inverta a ordem dos elementos de um vetor de 10 inteiros.
- ④ Identifique e armazene a posição do menor elemento em um vetor de 20 inteiros.
- ⑤ Preencha um vetor de 10 inteiros e, em seguida, ordene-o de forma crescente.

Assembly RISC-V

Exercícios 3



- 6 Preencha a diagonal principal de uma matriz 7x7 com um valor específico, deixando os outros elementos com valor 0.
- 7 Conte o número de elementos negativos em uma matriz 10x10.
- 8 Calcule a soma dos elementos das diagonais principal e secundária de uma matriz quadrada 5x5.
- 9 Preencha uma matriz 10 x 10 e, em seguida, zere todos os elementos abaixo da diagonal principal.
- 10 Encontre e retorne as coordenadas de um elemento específico dentro de uma matriz 8x8.

